Dukaan Pro - Database ERD (Version 2)

This version applies all MVP decisions we agreed: keep the full design, build in two releases, cancel-with-reason instead of editing, soft cash warnings, automatic numbers, and the seven corrections found while writing the workflows. PostgreSQL, one shared database, **shop_id on every table**.

Two releases, one database design

Release 1 (19 tables): shops, users, categories, brands, products, suppliers, purchases, purchase_items, supplier_payments, inventory_units, stock_levels, inventory_movements, customers, sales, sale_items, payments, expense_categories, expenses, cash_movements.
Release 1.1 (4 tables, dashed in the diagram): returns, return_items, refunds, cash_reconciliations. They are designed now, so adding them later is only a new migration.
Platform admin: the SaaS admin who creates shops is outside the shop tables. Keep a small separate platform_admins table (id, phone, password_hash) that is not part of the 23 shop tables.

How to read the diagrams

Symbol	Meaning
Bar --- crow's foot	One-to-many. Bar side = parent (1), crow's foot side = child (many).
Dashed line	Optional link (the foreign key may be empty).
Dashed table border, (R1.1)	Table is built in Release 1.1.
PK / FK / UQ	Primary key / Foreign key / Unique (UQ on a table means unique per shop: shop_id + column).
Orange bold name	New or changed in v2 (see the next page).
shop_id	Every table except shops has shop_id pointing to shops.id. Not drawn, to keep the picture clear.

Global design choices (unchanged)

- **IDs:** uuid.
- **Money:** bigint in paisa (Rs 100 = 10000).
- **Time:** timestamptz in UTC; business dates use date.
- **Enums:** text + CHECK so statuses can grow.
- **No delete:** master data uses is_active; money records are cancelled with a reason, never edited or deleted.
- **Computed, not stored:** paid amount, outstanding (Udhar), supplier payable, current cash and the customer ledger are calculated from records.

What changed from ERD v1 to v2

#	Change	Why
1	**Cancel with reason:** status + cancelled_at + cancel_reason on sales, purchases, payments, supplier_payments, expenses	Wrong entries are cancelled, not edited. History stays.
2	**cash_movements.reverses_movement_id** and type REVERSAL	A cancel adds an opposite cash row linked to the original. Nothing is deleted.
3	**cash_movements.is_overdraft**	Cash may go below zero after the owner confirms a warning. The row is marked so you can see it later.
4	**Removed sale_items.discount**	Discount is sale-level only. Profit math is simpler.
5	**shops.next_sale_no / next_purchase_no**, **sales.sale_no**, **purchases.purchase_no**	Automatic numbers (S-0001, P-0001), unique per shop.
6	**purchases.supplier_invoice_ref**	The supplier's own paper number is separate from our number.
7	**payments.receipt_group_id**	One Udhar payment can cover several sales (oldest first) and prints as one receipt.
8	**client_request_id** on sales, purchases, payments (unique per shop)	Double tap or bad network cannot create two sales. The same id returns the already-saved record.
9	**inventory_units.condition** (NEW / USED), status adds LOST and VOIDED	Used phones from walk-in sellers; lost stock; units of a cancelled purchase.
10	**suppliers.is_system**	Protects the default 'Walk-in seller' supplier.
11	**payments.customer_id** optional	Walk-in cash sales have no customer (v1 mistake).
12	**supplier_payments.purchase_id** optional	Allows paying a supplier's opening balance.
13	**purchases.is_opening_stock**	Enter existing stock without payable or cash movement.
14	**users.password_setup_token_hash** and expiry; password_hash optional until set	Owner sets his own password; admin-assisted recovery.
15	**Release split**	returns, return_items, refunds, cash_reconciliations move to Release 1.1 (still designed now).

Key constraints to enforce in PostgreSQL

Business rule	How to enforce	Where
IMEI unique	UNIQUE (shop_id, imei1) and (shop_id, imei2) where not null	DB
Login phone unique	UNIQUE (users.phone)	DB
One owner per shop	Partial unique index on users(shop_id) where role = OWNER	DB
Automatic numbers unique	UNIQUE (shop_id, sale_no), UNIQUE (shop_id, purchase_no). Counter is read and increased in the same transaction (UPDATE ... RETURNING)	DB + App
Double-save protection	UNIQUE (shop_id, client_request_id) on sales, purchases, payments. On conflict, return the existing record	DB + App
Names unique per shop	UNIQUE (shop_id, name) on categories, brands, expense_categories	DB
Quantities positive	CHECK (quantity > 0) on item tables	DB
Money never negative	CHECK (amount >= 0) on prices; > 0 on payments and cash movements	DB
No negative accessory stock	CHECK (quantity_on_hand >= 0)	DB
Phone sale item = 1 unit	CHECK: if inventory_unit_id is set then quantity = 1	DB
Cancelled rows have a reason	CHECK: status = CANCELLED requires cancelled_at and cancel_reason	DB
Reversal points to original	reverses_movement_id set only when type = REVERSAL; one reversal per movement (unique)	DB
A phone cannot be sold twice	Lock the unit row (FOR UPDATE), require IN_STOCK, set SOLD	App + DB
Phone sale needs a customer; Udhar needs a customer	Backend rejects otherwise	App
Payment <= outstanding	Check inside the payment transaction (lock the sale row). Outstanding counts only ACTIVE payments and non-cancelled sales	App
Supplier payment <= purchase left	Check inside the transaction	App
Cash below zero	Warn, require confirmation, then save with is_overdraft = true	App
Cancel a purchase	Blocked if any of its phones is not IN_STOCK or accessory stock would go negative	App
Cancel a sale	Phones back to IN_STOCK, accessory stock restored, active payments cancelled with cash reversal (owner confirms)	App
Only the owner can cancel	Role check on every cancel endpoint	App
Totals calculated by backend	Never trust totals from the client	App
One DB transaction per action	BEGIN ... COMMIT / ROLLBACK	App
Tenant isolation	shop_id from the session in every query. Extra safety: PostgreSQL Row Level Security	App + DB

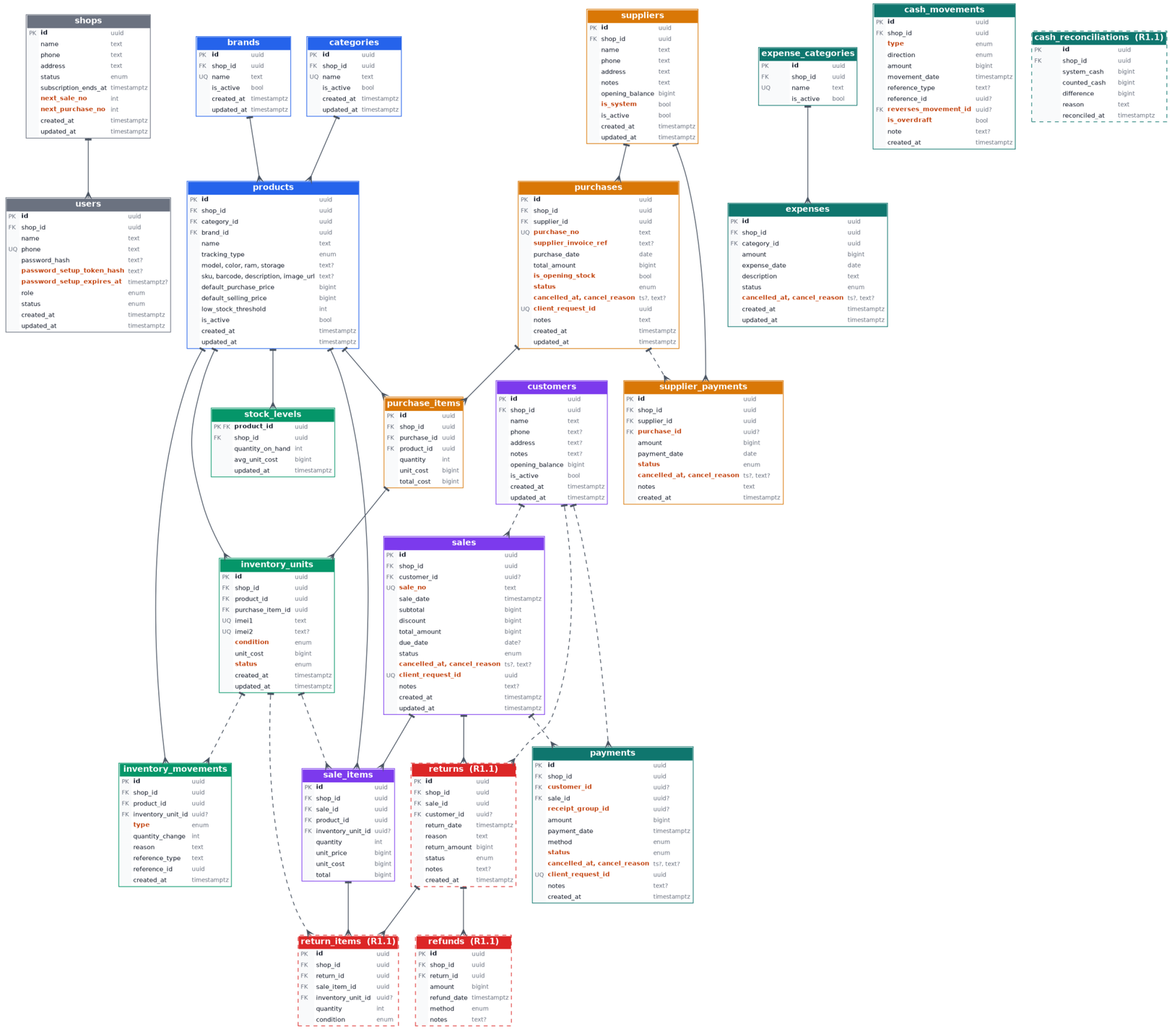
Suggested indexes

- (shop_id, date column) on sales, purchases, expenses, cash_movements.

- Join keys: sale_items(sale_id), purchase_items(purchase_id), payments(sale_id), payments(customer_id), inventory_units(product_id, status).
- Search: customers(shop_id, phone), products(shop_id, name), inventory_units(shop_id, imei1).
- Partial indexes on status = 'ACTIVE' / 'COMPLETED' for the dashboard queries.

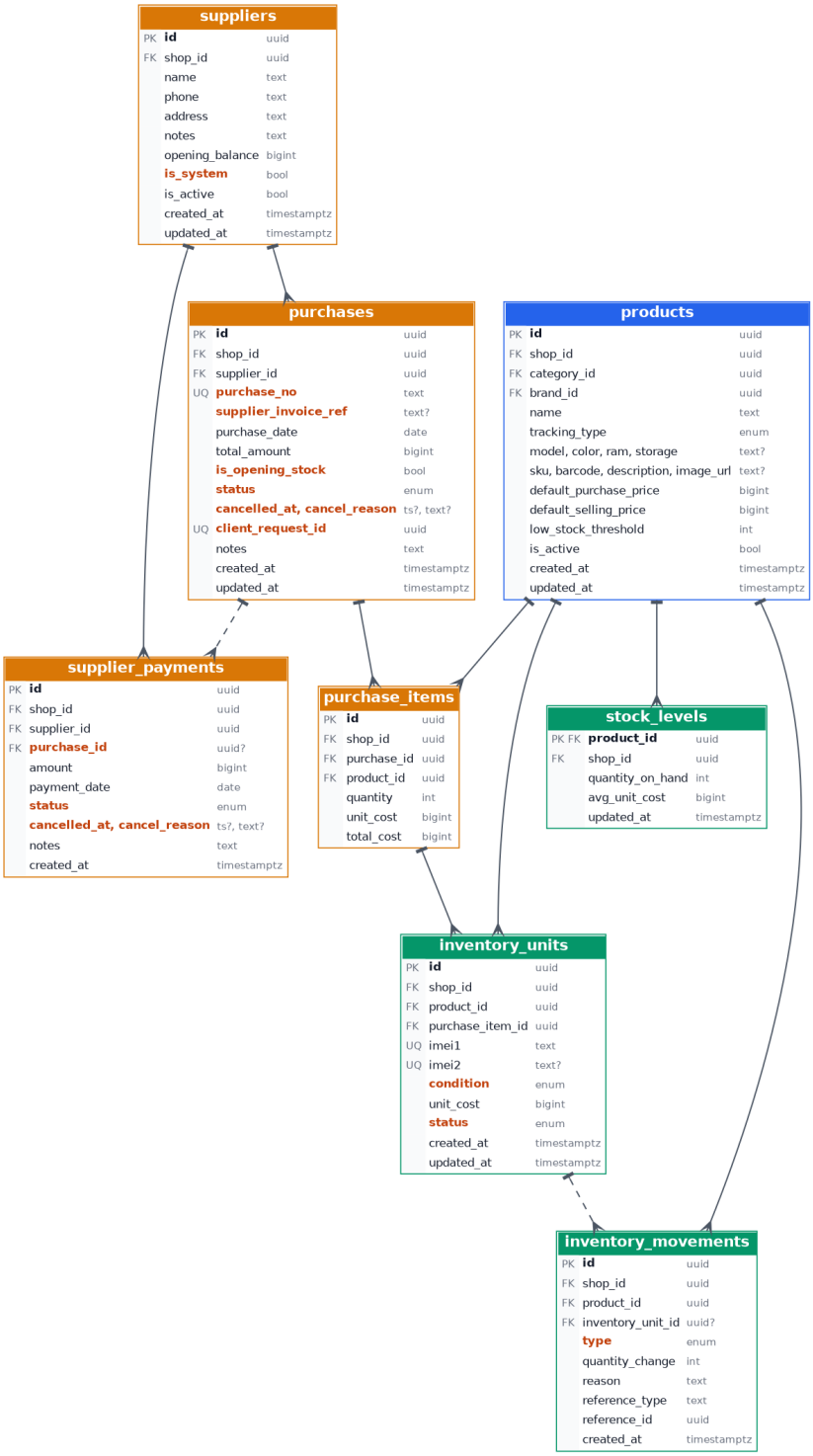
Full ERD v2 - 23 shop tables (19 in Release 1, 4 dashed = Release 1.1)

Every table except *shops* also has shop_id (FK to shops.id). Orange bold = new or changed in v2. Dashed line = optional link.



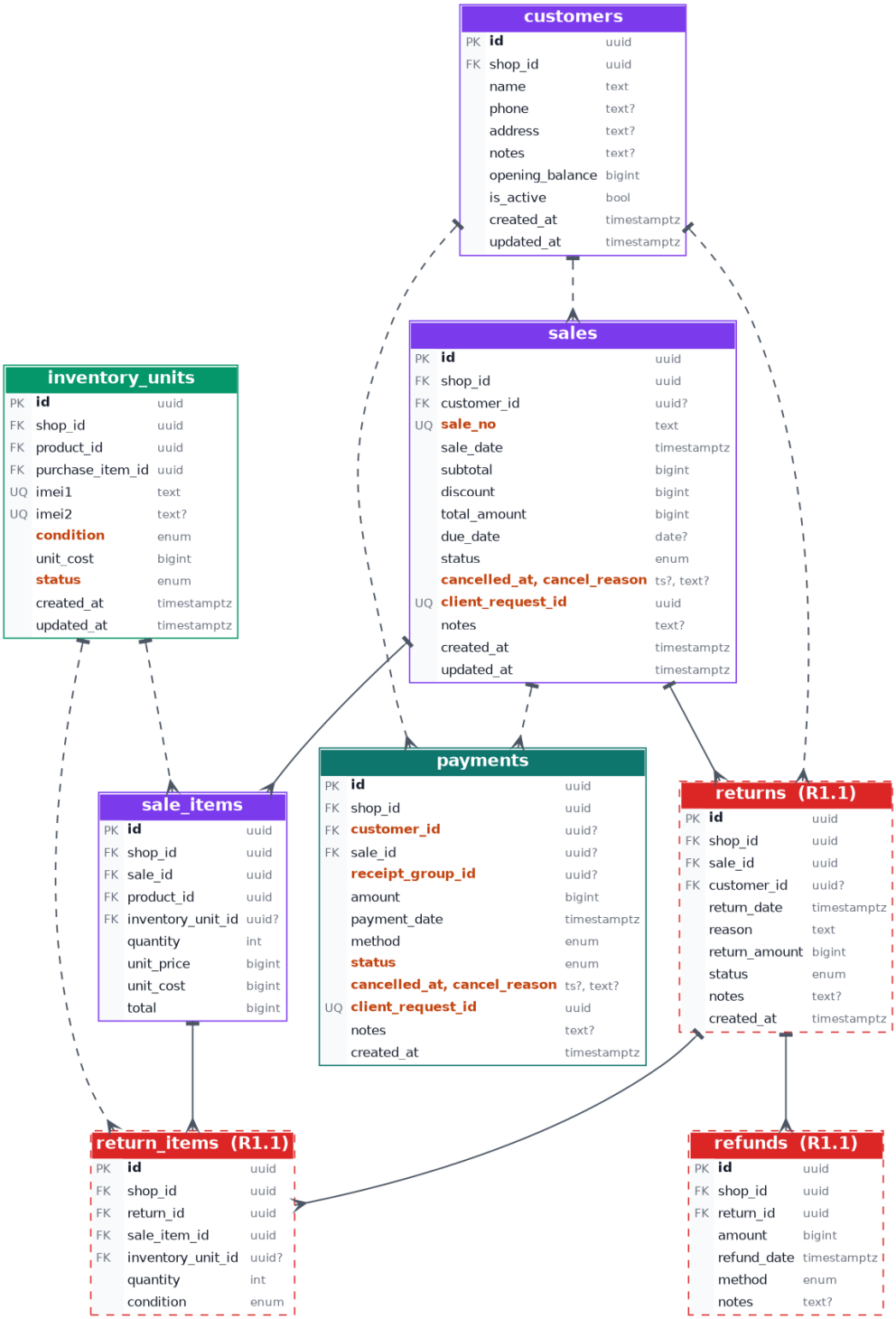
Focused view 1: Purchasing and Inventory

A purchase turns into stock. Phones become **inventory_units** (one row per IMEI, NEW or USED). Accessories update **stock_levels**. Every change is written to **inventory_movements**. supplier_payments is dashed to purchases because it can also pay an opening balance.



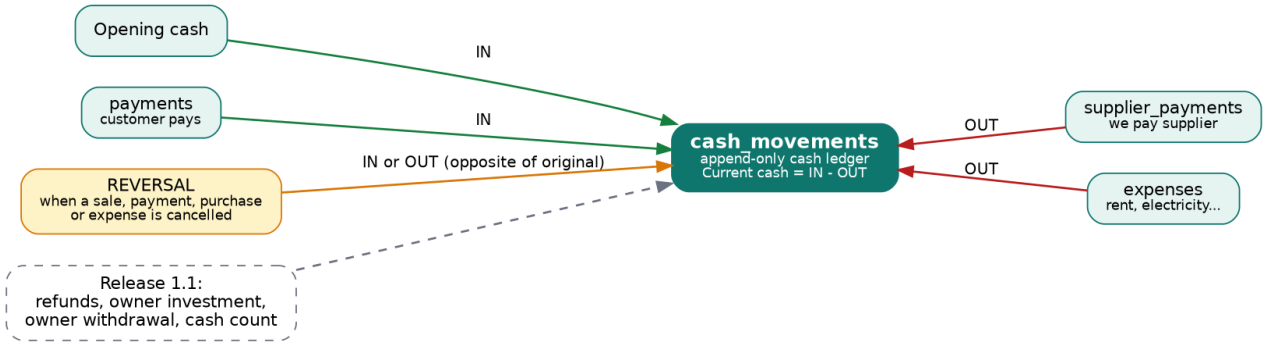
Focused view 2: Sales, Payments (Release 1) and Returns (Release 1.1)

A sale has items and many payments. In Release 1 a mistake is fixed with **Cancel sale**. Returns, return items and refunds (dashed) arrive in Release 1.1. payments.customer_id and sale_id are optional (walk-in sale, opening-balance payment).



Focused view 3: How cash flows

Every real money movement writes a row in **cash_movements**. A cancel writes a REVERSAL row pointing to the original. Sales and purchases alone never change cash; only payments do.



Example: Khalid sells a Rs 100,000 phone to Ali, Ali pays Rs 50,000, then it is cancelled by mistake

Step	Rows created / changed
1	Counter: shops.next_sale_no read and increased. sales (sale_no S-0002, total 100000, customer Ali, client_request_id) + sale_items (unit_price 100000, unit_cost 90000)
2	inventory_units: IN_STOCK to SOLD. inventory_movements: SALE, -1
3	payments: 50000 (ACTIVE). cash_movements: CUSTOMER_PAYMENT, IN, 50000
Result	Outstanding (Udhar) = 50000. Profit on the item = 10000. All in one DB transaction.
Cancel	sales.status = CANCELLED + reason. Unit back to IN_STOCK, movement SALE_CANCEL +1. payments.status = CANCELLED. cash_movements: REVERSAL, OUT, 50000 (reverses_movement_id = the original). Udhar and profit disappear; history stays.

Table dictionary

Columns marked V2 are new or changed in this version. shop_id on every table is a required foreign key to shops.id unless noted.

shops - Release 1

Column	Type	Key / note
id	uuid	PK
name	text	
phone	text	
address	text	
status	enum	
subscription_ends_at	timestamptz	
next_sale_no	int	**V2**
next_purchase_no	int	**V2**
created_at	timestamptz	
updated_at	timestamptz	

Rule: Top-level tenant. Status ACTIVE / SUSPENDED is separate from user status. The two counters create automatic numbers (S-0001, P-0001); they are increased inside the same transaction that saves the sale or purchase.

users - Release 1

Column	Type	Key / note
id	uuid	PK
shop_id	uuid	FK to shops
name	text	
phone	text	unique
password_hash	text?	
password_setup_token_hash	text?	**V2**
password_setup_expires_at	timestamptz?	**V2**
role	enum	
status	enum	
created_at	timestamptz	
updated_at	timestamptz	

Rule: Phone is unique across the platform (login). password_hash stays empty until the owner sets it through the one-time setup link (also used for admin-assisted recovery). MVP: one OWNER per shop.

categories - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
name	text	unique per shop
is_active	bool	
created_at	timestamptz	
updated_at	timestamptz	
Rule: Default categories are copied into each shop. Unique (shop_id, name).		

brands - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
name	text	unique per shop
is_active	bool	
created_at	timestamptz	
updated_at	timestamptz	
Rule: Shop-specific. Unique (shop_id, name).		

products - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
category_id	uuid	FK to categories
brand_id	uuid	FK to brands
name	text	
tracking_type	enum	
model, color, ram, storage	text?	optional
sku, barcode, description, image_url	text?	optional
default_purchase_price	bigint	
default_selling_price	bigint	
low_stock_threshold	int	
is_active	bool	
created_at	timestamptz	
updated_at	timestamptz	

Rule: tracking_type = QUANTITY or SERIALIZED and cannot change after the first transaction. Default prices are suggestions only. Never hard-deleted once used.

suppliers - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
name	text	
phone	text	
address	text	
notes	text	
opening_balance	bigint	
is_system	bool	**V2**
is_active	bool	
created_at	timestamptz	
updated_at	timestamptz	

Rule: is_system = true marks the default supplier 'Walk-in seller' (used phones bought from people). It cannot be deleted or renamed.

purchases - Release 1

Column	Type	Key / note
id	uuid	PK
shop_id	uuid	FK to shops
supplier_id	uuid	FK to suppliers
purchase_no	text	unique per shop ; **V2**
supplier_invoice_ref	text?	**V2**
purchase_date	date	
total_amount	bigint	
is_opening_stock	bool	**V2**
status	enum	**V2**
cancelled_at, cancel_reason	ts?, text?	**V2** ; optional
client_request_id	uuid	unique per shop ; **V2**
notes	text	
created_at	timestamptz	
updated_at	timestamptz	

Rule: purchase_no is automatic (P-0001). supplier_invoice_ref is the supplier's own paper number (optional). Paid amount is NOT stored; it is the sum of active supplier_payments. Opening-stock purchases create no payable and no cash movement. client_request_id stops double saves.

purchase_items - Release 1

Column	Type	Key / note
id	uuid	PK
shop_id	uuid	FK to shops
purchase_id	uuid	FK to purchases
product_id	uuid	FK to products
quantity	int	
unit_cost	bigint	
total_cost	bigint	

Rule: unit_cost is the real cost at that time and never changes. CHECK quantity > 0, unit_cost >= 0.

supplier_payments - Release 1

Column	Type	Key / note
id	uuid	PK
shop_id	uuid	FK to shops
supplier_id	uuid	FK to suppliers
purchase_id	uuid?	FK to purchases (optional) ; **V2**
amount	bigint	
payment_date	date	
status	enum	**V2**
cancelled_at, cancel_reason	ts?, text?	**V2** ; optional
notes	text	
created_at	timestamptz	

Rule: purchase_id is empty when paying the supplier's opening balance. Total active payments for a purchase cannot exceed its total_amount. A wrong payment is cancelled with a reason (cash reversal), not edited.

inventory_units - Release 1

Column	Type	Key / note
id	uuid	PK
shop_id	uuid	FK to shops
product_id	uuid	FK to products
purchase_item_id	uuid	FK to purchase_items
imei1	text	unique per shop
imei2	text?	unique per shop
condition	enum	**V2**
unit_cost	bigint	
status	enum	**V2**
created_at	timestamptz	
updated_at	timestamptz	

Rule: One row per physical phone. condition = NEW or USED. Status: IN_STOCK, SOLD, DAMAGED, LOST, VOIDED (VOIDED = purchase was cancelled). Only IN_STOCK can be sold. Unique (shop_id, imei1).

stock_levels - Release 1		
Column	**Type**	**Key / note**
product_id	uuid	PK ; FK to products
shop_id	uuid	FK to shops
quantity_on_hand	int	
avg_unit_cost	bigint	
updated_at	timestamptz	

Rule: Current quantity for QUANTITY products (chargers, covers). CHECK quantity_on_hand >= 0. Updated in the same DB transaction as every movement. avg_unit_cost is the simple average cost used for profit.

inventory_movements - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
product_id	uuid	FK to products
inventory_unit_id	uuid?	FK to inventory_units (optional)
type	enum	**V2**
quantity_change	int	
reason	text	
reference_type	text	
reference_id	uuid	
created_at	timestamptz	

Rule: Append-only history. Types: PURCHASE, SALE, SALE_CANCEL, PURCHASE_CANCEL, CUSTOMER_RETURN (R1.1), DAMAGED, LOST, ADJUSTMENT. Reason is required for adjustments, damage and loss.

customers - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
name	text	
phone	text?	
address	text?	
notes	text?	
opening_balance	bigint	
is_active	bool	
created_at	timestamptz	
updated_at	timestamptz	

Rule: Walk-in accessory sales use customer_id = NULL. A sale that contains a phone requires a customer (app rule).

sales - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
customer_id	uuid?	FK to customers (optional)
sale_no	text	unique per shop ; **V2**
sale_date	timestamptz	
subtotal	bigint	
discount	bigint	
total_amount	bigint	
due_date	date?	
status	enum	
cancelled_at, cancel_reason	ts?, text?	**V2** ; optional
client_request_id	uuid	unique per shop ; **V2**
notes	text?	
created_at	timestamptz	
updated_at	timestamptz	

Rule: sale_no is automatic (S-0001). Discount is sale-level only. total = subtotal - discount. Paid is NOT stored; it is the sum of active payments. If outstanding > 0 the customer is required. CANCELLED sales are ignored in Udhar, profit and reports.

sale_items - Release 1

Column	Type	Key / note
id	uuid	PK
shop_id	uuid	FK to shops
sale_id	uuid	FK to sales
product_id	uuid	FK to products
inventory_unit_id	uuid?	FK to inventory_units (optional)
quantity	int	
unit_price	bigint	
unit_cost	bigint	
total	bigint	

Rule: unit_price and unit_cost are saved at sale time. The old item-level discount column was removed (V2): discount is only on the sale. Phone = quantity 1 with inventory_unit_id. Accessory = quantity N, inventory_unit_id NULL. Selling below cost shows a warning but is allowed.

payments - Release 1

Column	Type	Key / note
id	uuid	PK
shop_id	uuid	FK to shops
customer_id	uuid?	FK to customers (optional) ; **V2**
sale_id	uuid?	FK to sales (optional)
receipt_group_id	uuid?	**V2**
amount	bigint	
payment_date	timestamptz	
method	enum	
status	enum	**V2**
cancelled_at, cancel_reason	ts?, text?	**V2** ; optional
client_request_id	uuid	unique per shop ; **V2**
notes	text?	
created_at	timestamptz	

Rule: customer_id is empty for a walk-in cash sale. sale_id is empty when paying old opening-balance Udhar. One Udhar payment can be split across several sales (oldest first); the rows share receipt_group_id so they print as one receipt. Payment cannot exceed outstanding.

expense_categories - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
name	text	unique per shop
is_active	bool	

Rule: Default categories (Rent, Electricity, Internet, Salary, Transport, Maintenance, Marketing, Other) copied per shop.

expenses - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
category_id	uuid	FK to expense_categories
amount	bigint	
expense_date	date	
description	text	
status	enum	
cancelled_at, cancel_reason	ts?, text?	**V2** ; optional
created_at	timestamptz	
updated_at	timestamptz	

Rule: Status ACTIVE or CANCELLED. Cancelled with a reason, never deleted. Net profit uses only ACTIVE expenses.

cash_movements - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
type	enum	**V2**
direction	enum	
amount	bigint	
movement_date	timestamptz	
reference_type	text?	
reference_id	uuid?	
reverses_movement_id	uuid?	FK to cash_movements (optional) ; **V2**
is_overdraft	bool	**V2**
note	text?	
created_at	timestamptz	

Rule: Append-only cash ledger. Types: OPENING_CASH, CUSTOMER_PAYMENT, SUPPLIER_PAYMENT, EXPENSE, REVERSAL, plus (Release 1.1) REFUND, OWNER_INVESTMENT, OWNER_WITHDRAWAL, RECONCILIATION_ADJUSTMENT. A cancel adds a REVERSAL row pointing to the original. is_overdraft marks a movement saved after the owner confirmed a 'cash will go below zero' warning.

returns - RELEASE 1.1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
sale_id	uuid	FK to sales
customer_id	uuid?	FK to customers (optional)
return_date	timestamptz	
reason	text	
return_amount	bigint	
status	enum	
notes	text?	
created_at	timestamptz	

Rule: RELEASE 1.1. Original sale is never edited. Reason is required. Until 1.1, mistakes are fixed with Cancel sale.

return_items - RELEASE 1.1

Column	Type	Key / note
id	uuid	PK
shop_id	uuid	FK to shops
return_id	uuid	FK to returns
sale_item_id	uuid	FK to sale_items
inventory_unit_id	uuid?	FK to inventory_units (optional)
quantity	int	
condition	enum	

Rule: RELEASE 1.1. condition = GOOD or DAMAGED. A serialized unit can be returned only once (unique on inventory_unit_id).

refunds - RELEASE 1.1

Column	Type	Key / note
id	uuid	PK
shop_id	uuid	FK to shops
return_id	uuid	FK to returns
amount	bigint	
refund_date	timestamptz	
method	enum	
notes	text?	

Rule: RELEASE 1.1. Money given back to the customer. Creates a cash OUT movement.

cash_reconciliations - RELEASE 1.1

Column	Type	Key / note
id	uuid	PK
shop_id	uuid	FK to shops
system_cash	bigint	
counted_cash	bigint	
difference	bigint	
reason	text	
reconciled_at	timestamptz	

Rule: RELEASE 1.1. Owner counts the drawer, the system compares and saves the difference with a reason.

Next steps

- Confirm the open questions with the pilot shop owner: staff helper, used phones, mobile load / SIM, language.

- Write the PostgreSQL / Prisma schema from this ERD (all 23 tables, but only wire up Release 1 screens first).
- Seed data: default categories, brands, expense categories, 'Walk-in seller' supplier.
- Then API design, and build in vertical slices: Auth, Products, Purchases, Inventory, Sales, Udhar, Expenses, Dashboard.